

DEPLOYMENT GUIDE

Crown Store PK

Complete Hostinger VPS Deployment — Frontend & Backend

Before you start

Next.js is a full-stack framework — your frontend pages and backend API routes live in one codebase and deploy together as a single app. There is no separate "deploy the backend" step.

This guide uses a Hostinger **VPS** (Ubuntu 22.04), not shared hosting — required because this app compiles a native database module and saves uploaded images to disk.

Why a VPS, not shared hosting

better-sqlite3

The database engine is a native module — it compiles specifically for your server. Shared hosting blocks this kind of server-side compilation.

Product image uploads

Admin-uploaded photos save directly to the server's disk. Shared/serverless hosting often doesn't persist files like this.

Buy a Hostinger VPS plan (KVM 1 or KVM 2 is enough), with **Ubuntu 22.04** as the OS.

What you need before starting

- ☐ A Hostinger VPS plan purchased, Ubuntu 22.04 selected
- ☐ A domain name (from Hostinger or anywhere)
- ☐ Your VPS IP address and root password (hPanel → VPS → Overview)
- ☐ This project folder on your own PC (crown-store-pk)
- ☐ (Recommended) A free GitHub account

Phase 1 — Put your code on GitHub (recommended)

This makes every future update as easy as one command. On your own PC, inside the project folder:

```
git init
git add .
git commit -m "Initial commit before going live"
```

Create a new **private** repository at github.com/new (don't add a README). Then run the commands GitHub shows you, which look like:

```
git remote add origin https://github.com/YOUR-USERNAME/crown-store-pk.git
git branch -M main
git push -u origin main
```

Don't want GitHub? Skip to Phase 4, Option B for direct file upload instead.

Phase 2 — Connect to your VPS

On your PC, open a terminal and run (replace with your real IP):

```
ssh root@YOUR_VPS_IP
```

Type **yes** when asked to continue connecting, then enter your VPS root password (from hPanel) — nothing shows on screen while typing, that's normal.

You're now inside your VPS. Every command from Phase 3 onward runs inside this SSH session unless noted otherwise.

Phase 3 — Install required software on the VPS

```
apt update && apt upgrade -y

curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

node -v
npm -v

apt install -y build-essential python3 git

npm install -g pm2

apt install -y nginx
```

Tool	Purpose
nodejs	Runs your Next.js app
build-essential, python3	Needed to compile better-sqlite3
git	Pulls your code from GitHub
pm2	Keeps the app running 24/7, auto-restarts on crash or reboot
nginx	Receives domain traffic and forwards it to the app

Phase 4 — Get your code onto the VPS

Option A — From GitHub

```
cd /var/www
git clone https://github.com/YOUR-USERNAME/crown-store-pk.git
cd crown-store-pk
```

Option B — Direct upload

In a **new terminal on your own PC** (keep the SSH one open), from inside your project folder:

```
scp -r . root@YOUR_VPS_IP:/var/www/crown-store-pk
```

Tip: delete the `node_modules` folder on your PC first — it gets reinstalled fresh on the server in Phase 6, and skipping it avoids uploading thousands of unnecessary files.

Phase 5 — Bring your real data over (optional)

If you've already customized products/settings locally and want that live, copy your database from your own PC:

```
scp "C:\Users\NINJA\Desktop\crown-store-pk\data\crown-store.db"  
root@YOUR_VPS_IP:/var/www/crown-store-pk/data/
```

Skipping this is fine — the app auto-creates a fresh database with 4 demo products and a working admin account on first run.

Phase 6 — Install dependencies and build

```
cd /var/www/crown-store-pk  
npm install
```

If you see a `better-sqlite3` error, run this then continue:

```
npm rebuild better-sqlite3
```

Build the production app (compiles frontend pages and backend API routes together):

```
npm run build
```

Phase 7 — Security: set a real login secret

The app currently falls back to a default hardcoded secret for signing login sessions. Set your own before going live:

```
nano /var/www/crown-store-pk/.env
```

Type this line (use your own long random string, 40+ characters, never share it):

```
JWT_SECRET=put-a-long-random-string-here-never-share-it
```

Save: `Ctrl+X`, then `Y`, then `Enter`.

Phase 8 — Start the app with PM2

```
cd /var/www/crown-store-pk
pm2 start npm --name "crown-store" -- start
pm2 save
pm2 startup
```

`pm2 startup` prints another command — copy and run exactly what it shows you (makes the app auto-start after a VPS reboot).

```
pm2 status
```

You should see **crown-store** with status **online**.

Phase 9 — Configure Nginx

```
nano /etc/nginx/sites-available/crown-store
```

Paste this in (replace `yourdomain.com` in both places):

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Save (`Ctrl+X`, `Y`, `Enter`), then activate it:

```
ln -s /etc/nginx/sites-available/crown-store /etc/nginx/sites-enabled/  
nginx -t  
systemctl restart nginx
```

`nginx -t` should say "syntax is ok". If it errors, stop and share the exact message.

Phase 10 — Point your domain at the VPS

Hostinger hPanel → Domains → your domain → DNS / Name Servers → DNS Zone Editor. Add:

Type	Name	Points to	TTL
A	@	YOUR_VPS_IP	14400
A	www	YOUR_VPS_IP	14400

DNS can take 15 minutes to 24 hours to propagate — check progress at dnschecker.org. Once propagated, `http://yourdomain.com` should load your site (no padlock yet).

Phase 11 — Add free SSL (https padlock)

Once DNS has propagated:

```
apt install -y certbot python3-certbot-nginx  
certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Enter an email (for renewal reminders) and agree to the terms. Certbot updates Nginx automatically and silently renews the certificate every ~90 days on its own.

`https://yourdomain.com` should now load with a padlock icon.

Phase 12 — Go-live checklist

- ☐ Visit `/login` and sign in with `admin@crowncore.pk` / `admin123` to confirm the admin panel works live
- ☐ Message Claude before going public — there's no "change password" button yet, and the default password should be changed first
- ☐ Enter real SMTP email details in Admin → Settings so order-notification emails actually send
- ☐ Set your real COD charges / announcement bar text in Admin → Settings
- ☐ Place one real test order end-to-end (cart → checkout → confirmation)
- ☐ Delete that test order from Admin → Orders afterward

How to push future updates

If you used GitHub

On your PC:

```
git add .
git commit -m "describe what changed"
git push
```

On the VPS:

```
cd /var/www/crown-store-pk
git pull
npm install
npm run build
pm2 restart crown-store
```

If you used direct upload

Repeat the `scp -r .` command from Phase 4 Option B, then on the VPS:

```
npm install
npm run build
pm2 restart crown-store
```

Backups (do this weekly)

Your real data is not on GitHub. Orders, products, and customer accounts live in one file on the VPS: `data/crown-store.db`. Back it up regularly.

On your PC:

```
scp root@YOUR_VPS_IP:/var/www/crown-store-pk/data/crown-store.db ./backup-crown-store-DATE.db
```

Also periodically back up `public/images/products/` if you've uploaded many product photos through the admin panel.

Troubleshooting / useful commands

<code>pm2 logs crown-store</code>	See live app errors/output
-----------------------------------	----------------------------

<code>pm2 restart crown-store</code>	Restart the app
<code>pm2 status</code>	Check if it's running
<code>systemctl status nginx</code>	Check if Nginx is healthy
<code>nginx -t</code>	Check Nginx config for typos
<code>certbot renew --dry-run</code>	Test that SSL auto-renewal works

If you get stuck, copy the **exact** error message and send it over — don't guess or skip ahead. Deployment errors are almost always one specific, fixable thing.